

Shortest Path for Autonomous Navigation: **A Comparative Study of *Quantum Hybrid*** **(QAOA) vs. *Classical Approaches***

22BAI1095 – Shireen Aliza Ali

Problem Statement & Objective

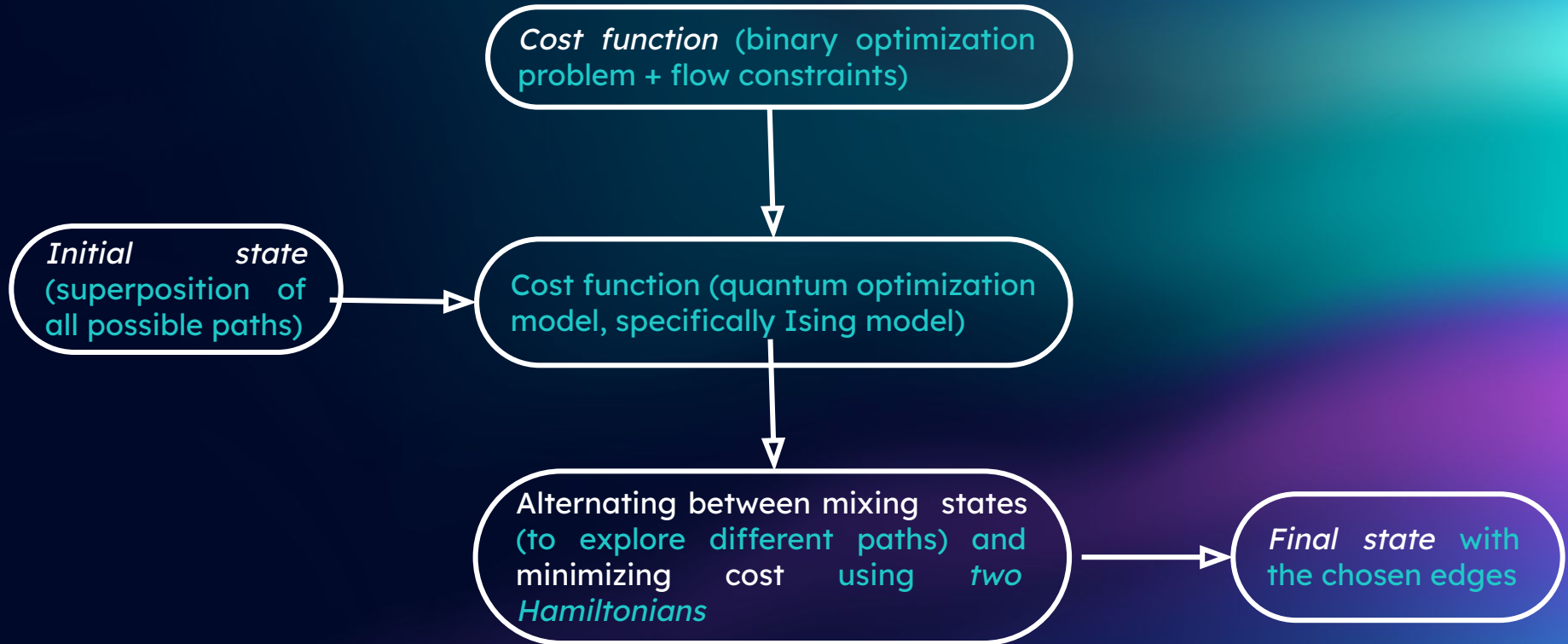
Challenge: Efficient path planning

Limitations of Classical Methods: *Classical Algorithms* struggle with large-scale problems and real-time constraints.

Potential of Quantum Computing: *QAOA* offers a novel approach and features parallelism but lacks comparative studies in drone navigation.

Objective: Compare **QAOA** with **Classical Algorithms** in a simulated drone environment based on speed, optimality, and scalability.

Methodology



Dijkstra's Algorithm

Guaranteed Optimality: Explores **all nodes** to ensure the shortest path is found.

Exhaustive Exploration: No reliance on heuristics; systematically evaluates every possibility.

```
import heapq

def dijkstra(graph, start):
    pq = [(0, start)]
    shortest_paths = {start: 0}
    visited = set()

    while pq:
        (cost, node) = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)

        for neighbor, weight in graph[node]:
            new_cost = cost + weight
            if neighbor not in shortest_paths or new_cost < shortest_paths[neighbor]:
                shortest_paths[neighbor] = new_cost
                heapq.heappush(pq, (new_cost, neighbor))

    return shortest_paths
```

Greedy algorithm using a **priority queue (min-heap)**.

Time Complexity: $O((V+E)\log V)$.

Iterates through neighbors, updating shortest paths dynamically.

A* Algorithm

Uses a **heuristic function** for faster convergence.

More efficient than Dijkstra when heuristic is accurate.

```
import heapq

def heuristic(node, goal):
    return abs(node - goal) # Example heuristic

def a_star(graph, start, goal):
    pq = [(0, start)]
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0

    while pq:
        current_cost, current_node = heapq.heappop(pq)
        if current_node == goal:
            return g_score

        for neighbor, weight in graph[current_node]:
            tentative_g = g_score[current_node] + weight
            f_score = tentative_g + heuristic(neighbor, goal)

            if tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                heapq.heappush(pq, (f_score, neighbor))

    return g_score
```

Bellman-Ford Algorithm

Handles negative weight edges.

Slower than Dijkstra due to its iterative nature.

```
def bellman_ford(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0

    for _ in range(len(graph) - 1):
        for node in graph:
            for neighbor, weight in graph[node]:
                if distances[node] + weight < distances[neighbor]:
                    distances[neighbor] = distances[node] + weight

    return distances
```

Iterative dynamic programming approach.

BFS Algorithm

Explores nodes level by level.

Does not consider weights.

```
def bfs(graph, start, goal):
    queue = deque([(start, [start])])

    while queue:
        node, path = queue.popleft()
        if node == goal:
            return path

        for neighbor, _ in graph.get(node, []):
            queue.append((neighbor, path + [neighbor]))

    return None
```

DFS Algorithm

Uses recursion to explore paths deeply.

Inefficient for shortest path finding.

```
def dfs(graph, start, goal, path=None):
    if path is None:
        path = [start]

    if start == goal:
        return path

    for neighbor, _ in graph.get(start, []):
        if neighbor not in path:
            result = dfs(graph, neighbor, goal, path + [neighbor])
            if result:
                return result

    return None
```

Quantum Approach – QAOA

Converts the problem into a **QUBO (Quantum Unconstrained Binary Optimization)** format.

Uses a **parameterized quantum circuit** to find an approximate solution.

Advantage: Might scale better for large graphs.

```
from qiskit import Aer, execute
from qiskit.circuit.library import QAOAAnsatz
from qiskit.optimization.applications.ising import max_cut

# Define graph as adjacency matrix
graph = {
    0: [(1, 2), (2, 3)],
    1: [(0, 2), (2, 1)],
    2: [(0, 3), (1, 1), (3, 4)],
    3: [(2, 4)]
}

# Convert to Ising Hamiltonian for QAOA
qubo = max_cut.get_operator(graph)
qaoa = QAOAAnsatz(qubo, reps=1)

# Simulate on quantum backend
backend = Aer.get_backend('qasm_simulator')
result = execute(qaoa, backend, shots=1000).result()
counts = result.get_counts()
```

Executes on IBM Qiskit quantum simulator.

Transforms graph problem into a **quantum optimization problem**.

Running QAOA on IBM Cloud

Quantum execution can be done on local simulators or IBM cloud.
Real quantum machines introduce errors & delays.

```
from qiskit import IBMQ

IBMQ.load_account()
provider = IBMQ.get_provider('ibm-q')
backend = provider.get_backend('ibmq_qasm_simulator')

qaoa_job = execute(qaoa, backend, shots=1000)
qaoa_result = qaoa_job.result()
counts = qaoa_result.get_counts()
```

Observations:

IBM Cloud runs real quantum circuits but introduces noise.
Slower than local simulators due to network overhead.

Local Execution vs. IBM Hardware Execution

Classical algorithms remain the best choice for pathfinding on small graphs.

IBM Hardware provided a noticeable performance boost for QAOA but was unnecessary for classical algorithms.

Algorithm Comparison:

Algorithm	Time (s)	Total Cost	Path
Dijkstra	0.0021	4.5000	[0, 2, 3]
A*	0.0010	4.5000	[0, 2, 3]
Bellman-Ford	0.0000	4.5000	[0, 2, 3]
BFS	0.0000	5.4000	[0, 1, 3]
DFS	0.0000	5.4000	[0, 1, 3]
QAOA	4.2450	4.5000	[0, 2, 3]

Local Execution

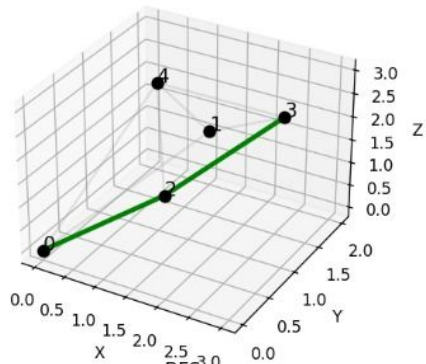
Algorithm Comparison:

Algorithm	Time (s)	Total Cost	Path
Dijkstra	0.0010	4.5000	[0, 2, 3]
A*	0.0000	4.5000	[0, 2, 3]
Bellman-Ford	0.0010	4.5000	[0, 2, 3]
BFS	0.0000	5.4000	[0, 1, 3]
DFS	0.0000	5.4000	[0, 1, 3]
QAOA	20.4153	4.5000	[0, 2, 3]

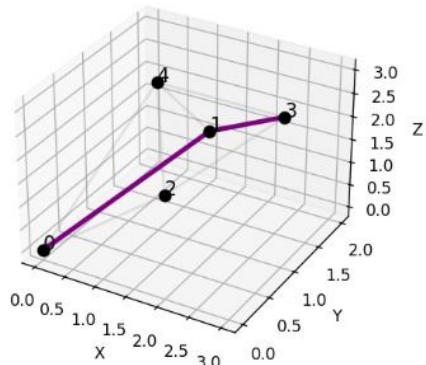
IBM Hardware Execution

Results

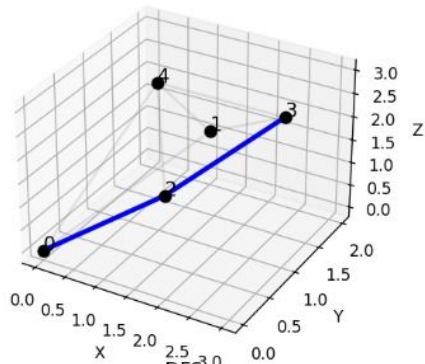
Dijkstra
Cost: 4.50, Time: 0.0010s



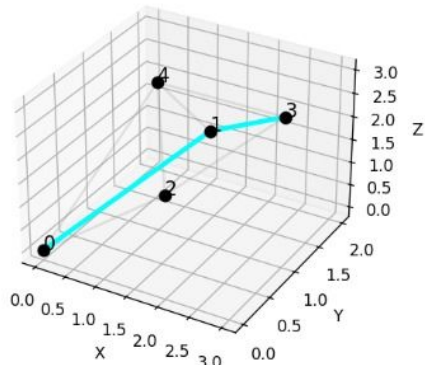
BFS
Cost: 5.40, Time: 0.0000s



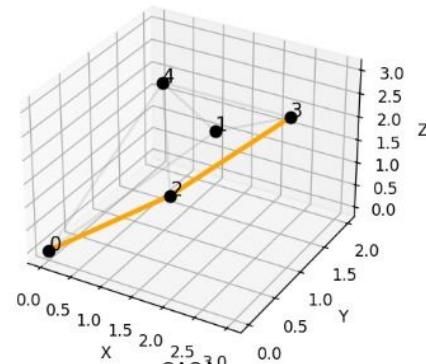
A*
Cost: 4.50, Time: 0.0000s



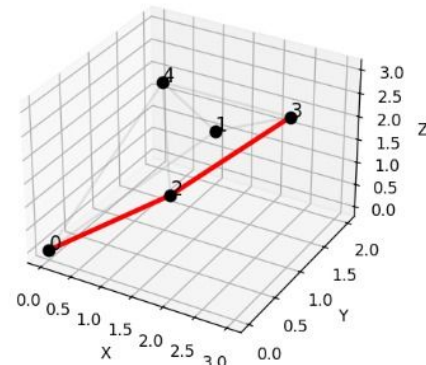
DFS
Cost: 5.40, Time: 0.0000s



Bellman-Ford
Cost: 4.50, Time: 0.0010s



QAQA
Cost: 4.50, Time: 20.8786s



Observations

Classical Algorithms (Dijkstra, A*, Bellman-Ford, BFS, DFS) show similar execution times on both local and IBM hardware, demonstrating their efficiency.

A* was the fastest among them.

BFS and DFS found suboptimal paths (cost = 5.40).

Quantum Algorithm (QAOA)

Significantly slower on local execution (20.8786s).

Much faster on IBM Hardware (4.5s), showing quantum speed-up potential.

Still needs optimization to be competitive with classical algorithms for this problem size.